

ДЕТАЛНО РЕЗИМЕ И ЗАКЛУЧОЦИ ОД ИСТРАЖУВАЊЕТО

1. Вовед

- Овој проект истражува како вештачката интелигенција може да помогне во оптимизација на складирањето и продажбата на фотоволтаичната (PV) енергија на европските пазари на електрична енергија. Со комбинирање на историски пазарни цени, временски податоци и податоци за PV производство од 6 европски земји, проектот е скрукуриран околу четири истражувачки прашања:

- **RQ1** - Колку прецизно можеме да ги предвидиме идните цени на електричната енергија?
- **RQ2** - Колку прецизно можеме да го предвидиме идното PV производство?
- **RQ3** - Која стратегија за складирање/продажба на енергија го максимизира приходот врз основа на тие предвидувања?
- **RQ4** - Колку е чувствителна таа стратегија на грешки во предвидувањата и во кои услови грешките чинат најмногу?

- Секое прашање се гради врз претходното, односно, предвидувањата од RQ1 и RQ2 директно се користат како влез во RQ3, а RQ4 ја тестира робусноста на целиот систем.

- Преку овој документ детално го опишуваме целиот процес, почнувајќи од избор и подготовка на податоците, моделирање, резултати и на крајот финалните заклучоци.

2. Физика на фотоволтаичното производство

- Колку струја произведува еден соларен панел зависи од тоа колку сончева светлина паѓа врз него. Тоа зависи од неколку работи:

1. Под кој агол е поставен панелот - панелите обично се наведнати под одреден агол за да фаќаат што повеќе Сонце преку целата година.

2. На која страна гледа панелот - панел кој е свртен на југ фаќа најмногу сонце, а панел кој е свртен на исток/запад фаќа Сонце порано/подоцна во денот соодветно.

3. Каде е Сонцето во моментот - ова зависи од часот (пониско е наутро и навечер, највисоко е напладне) и од сезоната (лето е повисоко, зима е пониско).

4. Колку е голем панелот - поголем панел значи повеќе струја, но не го менува распоредот на производство преку денот.

- Во нашиот проект, согласно податоците кои ги најдовме за изработка на овој проект, тие се збир од сите соларни панели во таа одредена земја (илјадници панели, секој со свој агол и насока). Не постои еден агол за цела земја бидејќи таквите информации не беа јавно достапни.

- Поради тоа, користиме веќе измерено сончево зрачење (shortwave_radiation), бројка која веќе го содржи ефектот на Сонцето и времето. Дополнително, часот и месецот (hour, month) му помагаат на моделот да го научи дневниот и сезонски шаблон кога Сонцето е посилено/послабо.

3. Наоѓање и избор на податочни множества

- За целите на истражувањето ни беа потребни три категории на податоци: **историски пазарни цени на електрична енергија, податоци за фотоволтаично производство и метеоролошки податоци**, сите на **часовно** ниво, за повеќе европски земји.

- Земји опфатени во истражувањето: **Германија, Грција, Холандија, Полска, Шпанија Шведска.**

- Изборот на овие шест земји е направен со цел да се опфати разновидна географска и климатска распределба на сочевото зрачење низ Европа, па заедно со тоа и различни режими на PV производство и пазарно однесување:

1. **Јужни/сончеви пазари** - Шпанија и Грција (високо и стабилно PV производство)

2. **Централноевропски пазари** - Германија и Полска (умерено производство, поголема варијабилност)

3. **Западноевропски пазар** - Холандија (послабо сончево зрачење)

4. **Северен пазар** - Шведска (најмалку сончево зрачење)

- Извори на податоци:

Категорија	Извор	Начин на преземање
Цени на електрична енергија	Day-ahead пазарни цени (EUR/MWh)	Готови историски CSV фајлови по земја
PV производство	ENTSO-E Transparency Platform	API преку entso-e пакет
Метеоролошки податоци	Open-Meteo Historical Forecast API	REST API повици по фиксни координати за секоја земја

- **Временски опфат:** секое податочно множество опфаќа период од 4 години на часовно ниво (**јануари 2022 - декември 2025**), што резултира со 34 872 часовни записи по земја.

- **Причина за day-ahead цени:** day-ahead пазарот е место каде производителите и трговците ги затвораат позициите за наредниот ден, еден ден однапред, па тој хоризонт на одлучување е релевантен за прашањето на овој проект (“кога да се продаде/складира произведената енергија”).

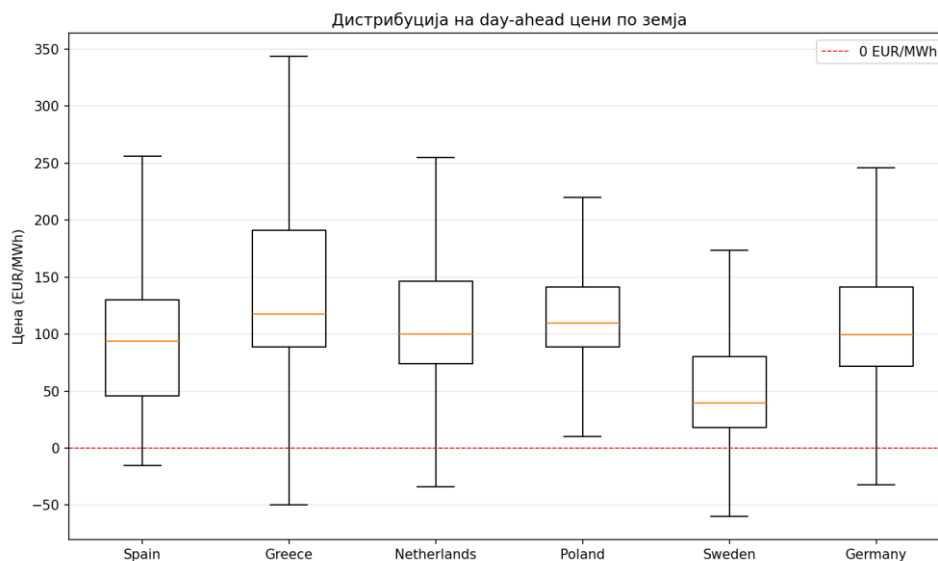
- **Спојување на изворите:** сите три извори се спојуваат по заеднички **datetime** во една единствена табела по земја, каде секој ред е еден час со сите релевантни информации заедно. Оваа споена табела е основата врз која подоцна се гради feature engineering (lag-ови, календарски карактеристики, хоризонт-специфички таргети).

4. Exploratory Data Analysis (EDA)

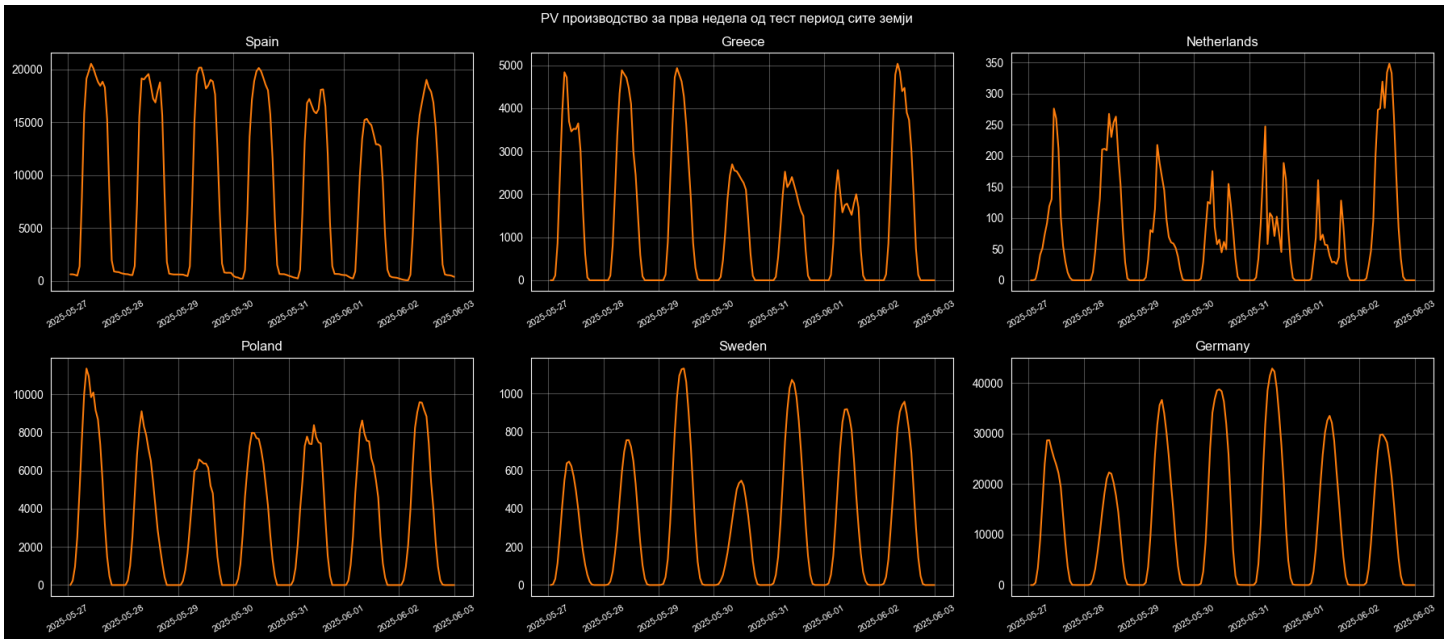
- Пред да продолжиме со feature engineering и моделирање, направивме основна истражувачка анализа за да ги разбереме карактеристиките на секој пазар, за да знаеме што би можеле да очекуваме понатаму.

Земја	Просечна цена	Ст. девијација (цена)	% часови со негативна цена	Просечно производство	Максимално производство
Шпанија	95.4	67.2	2.3%	4,907.8	23,999
Грција	150.7	107.4	0.4%	1,080.3	6,816
Холандија	125.3	103.7	4.1%	58.4	427.7
Полска	119.8	63.1	1.6%	1,693.5	13,682.3
Шведска	62.4	73.5	3.8%	104	1,200.3
Германија	124.7	105	3.3%	7,146.7	52,466.2

- Тука се видоа неколку разлики помеѓу земјите: **Германија и Шпанија имаат огромно поголемо производство од Холандија и Шведска, што е логично бидејќи се поголеми земји со поголем PV капацитет. Шведска има најниска медијана на цена, но релативно голема дистрибуција (веројатно поради силна зависност од хидро/ветер производство). Негативни цени** (кога произведувачите плаќаат за да се ослободат од вишок енергија) се појавуваат кај сите земји, но најчесто кај Холандија.

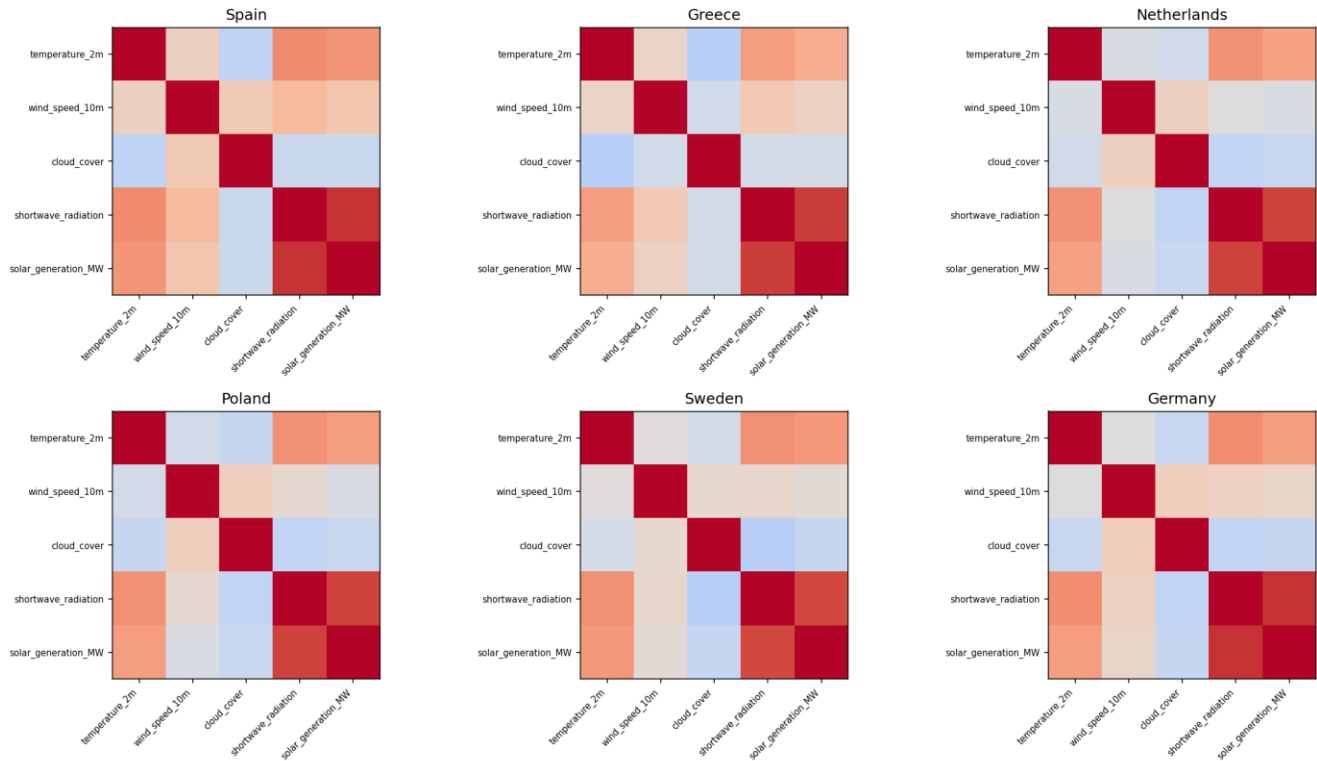


- **PV производството** следи строг “on/off” дневен циклус - секој ден оди од нула (ноќ) до пикот (напладне) и назад повторно. Ова го објаснува **U-обликот во Baseline моделот во RQ2**.

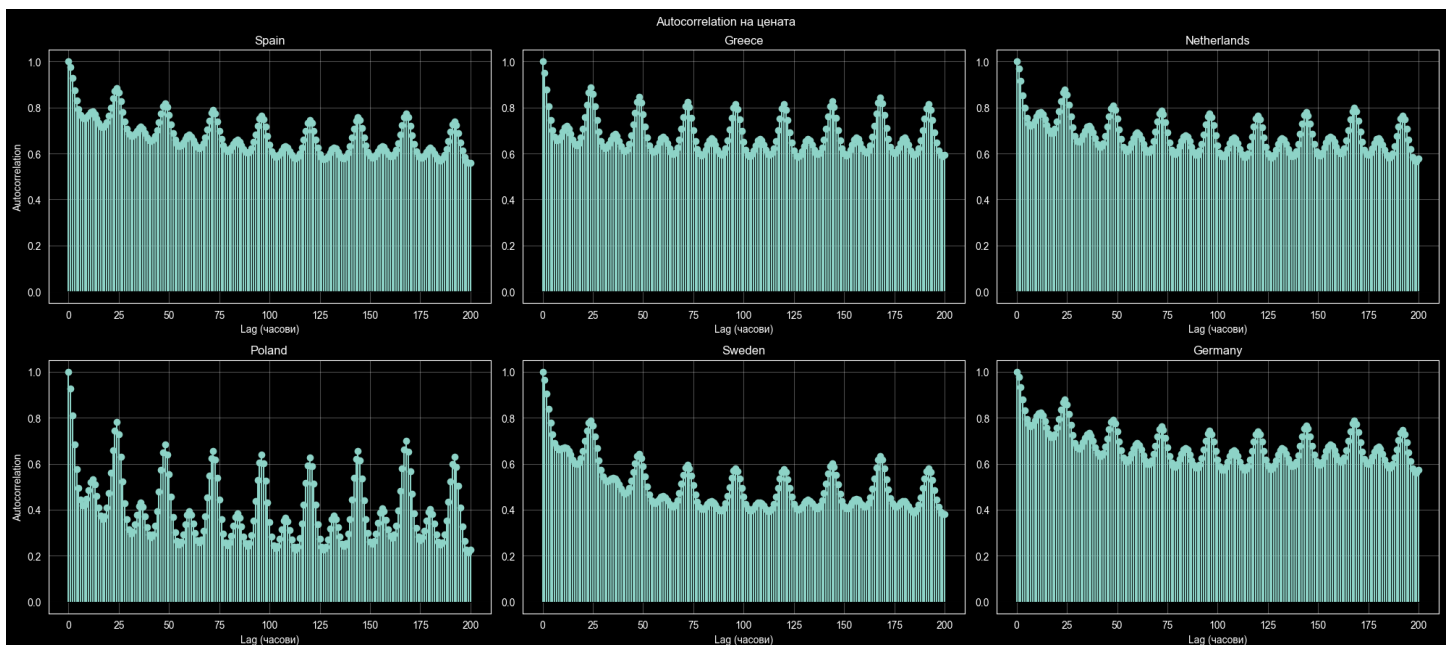


- Корелационата анализа помеѓу временските услови и производството покажа конзистентност кај сите 6 земји. Имаме **позитивна силна врска** помеѓу сончевото зрачење и производството и **негативна врска** со облачноста, што е очекувано и потврдено пред моделирање.

Корелација: временски услови vs PV производство



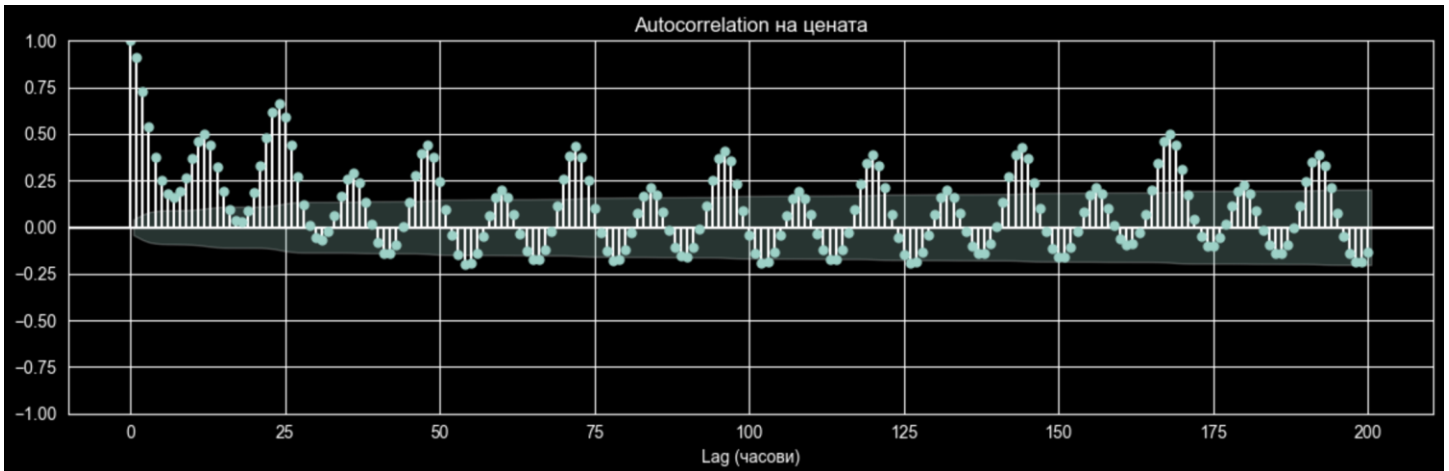
- Autocorrelation анализата на цената покажа **јасни пикови на секои 24 часа** (дневен циклус), што е директна основа зошто избравме **lag features (1h/24h/168h)**.



5. Feature Engineering

- Откако ги споивме трите извори во единствена табела (**datetime + цена + производство + временски услови**), суровите податоци сами по себе не се доволни за да можеме да предвидуваме нешто, затоа **од овие сурови податоци изведовме дополнителни карактеристики кои носат времеска информација**.

1. **Lag features** (историски вредности): за цената и производството создадовме по три поместени (lag) верзии, вредноста **пред 1 час, пред 24 часа и пред 168 часа**. Изборот за овие три вредности не е случаен, туку е осознаен преку **autocorrelation анализа**, која покажа јасни пикови точно на 24-часовни (**дневен циклус**), и 168-часовни (**неделен циклус**) интервали.



```
11 df["price_lag_1h"] = df["Price (EUR/MWhe)"].shift(1)
12 df["price_lag_24h"] = df["Price (EUR/MWhe)"].shift(24)
13 df["price_lag_168h"] = df["Price (EUR/MWhe)"].shift(168)
14
15 df["solar_lag_1h"] = df["solar_generation_MW"].shift(1)
16 df["solar_lag_24h"] = df["solar_generation_MW"].shift(24)
17 df["solar_lag_168h"] = df["solar_generation_MW"].shift(168)
```

2. **Target features** (идни вредности за предвидување): со **.shift(-h)** - негативен поместувач кој гледа напред во иднината, но во спротивна насока од lag-овите (кои гледаат назад кон минатото) **се создаваат 24 таргет колони за секој хоризонт**.

```
24 for h in range(1, 25):
25     df[f"target_h{h}"] = df["Price (EUR/MWhe)"].shift(-h)
26     df[f"target_pv_h{h}"] = df["solar_generation_MW"].shift(-h)
27
28 df = df.dropna().reset_index(drop=True)
```

- ова е **методолошки избор**, наместо еден модел кој предвидува само “следниот час”, градиме посебни таргети за секој од 24-те часа однапред, што ни овозможува тренирање на модели кои се хоризонт-специфични (**одделен модел за h=1, за h=2, и така се до h=24**) - секој специјализиран точно за тоа колку часа однапред гледа.

3. Календарски карактеристики: директно извлечени од `datetime` колоната, `hour` (час од денот, 0-23), `day_of_week` (ден од неделата), `month` (месец 1-12), `is_weekend` (дали е сабота/недела). Овие карактеристики му помагаат на моделот да го научи дневниот и неделен шаблон на потрошувачката/производството (пример: цените се пониски навечер за време на викенд).

```
19 df["hour"] = df["datetime"].dt.hour
20 df["day_of_week"] = df["datetime"].dt.dayofweek
21 df["month"] = df["datetime"].dt.month
22 df["is_weekend"] = df["day_of_week"].isin([5, 6]).astype(int)
```

- **Чистење на рабови (`dropna()`):** по создавање на `lag/target` колоните, првите 168 реда и последните 24 реда содржат празни (NaN) вредности, па затоа тие редови ги отфрливме, со што табелата ни е чиста со 34 872 часа по земја.

```
28 df = df.dropna().reset_index(drop=True)
```

6. Train / Validation / Test поделба

- Пред тренирање на било кој модел, целата табела ја делиме на **три последователни, непреклопувачки периоди (70%, 15%, 15%)**:

```
11 train_end = int(len(df) * 0.7)
12 val_end = int(len(df) * 0.85)
13 test = df.iloc[val_end:]
```

- Ова е клучна методолошка одлука која што спречува **data leakage**. Во нашиов случај, кога некој тргува со електрична енергија во понеделник, тој физички не може да види податоци од среда истата недела, бидејќи тие се уште не постојат. Ако податоците ги мешавме случајно пред поделбата, моделот ќе можеше случајно да заврши со тренирачки редови кои се временски веднаш до тестирачки редови (пример: часот 14:00 во тренирачкото множество, а часот 15:00 во тестирачко множество). **Бидејќи цените и производството се силно автокорелирани од час до час, ова го олеснува проблемот да добиеме нереално висок резултат кој нема реална примена (data leakage).**

- **Дополнителна превенција од leakage:** капацитетот на батеријата (RQ3) и праговите во rule-based (25-ти и 75-ти перцентил на цена) се пресметуваат исклучиво од `train` периодот, не од `test`, инаку MPC би знаел однапред каква е распределбата на цените во иднина.

7. Скалирање на податоците

- Невронските мрежи (LSTM/TFT) работат многу подобро кога влезните вредности се во слична скала, за разлика од tree-based моделите (XGBoost) кои не се чувствителни на скалата на податоците.

```
30 scaler_x = StandardScaler().fit(df[features].iloc[:train_end])
31 scaler_y = StandardScaler().fit(df[[target]].iloc[:train_end])
32
33 X_all = scaler_x.transform(df[features])
34 y_all = scaler_y.transform(df[[target]]).ravel()
```

- Исто така, за повторно да избегнеме data leakage, **scaler-от пресметува (fit) само од тренирачкото множество**. Потоа тој пресметан scaler се применува (transform) на целата табела, вклучувајќи ги валидацискиот и тестирачкиот дел. **Ова е важно бидејќи ако дозволиме scaler-от да види статистики (просек/девијација) од тест периодот, моделот индиректно ќе добиеше информација за рапределбата на идните податоци.**

- Откако заврши предвидувањето, **резултатите ги вративме назад во оригинална скала** за да можат MAE/RMSE да се читаат во реални единици (EUR/MWh или MW).

```
82 y_pred = scaler_y.inverse_transform(y_pred_scaled.reshape(-1, 1)).ravel()
83 y_true = scaler_y.inverse_transform(y_test.numpy().reshape(-1, 1)).ravel()
84 y_train_pred = scaler_y.inverse_transform(y_train_pred_scaled.reshape(-1, 1)).ravel()
85 y_train_true = scaler_y.inverse_transform(y_train.numpy().reshape(-1, 1)).ravel()
```

8. Чистење и валидација на суровите податоци

- Суровите податоци за PV производството кои ги преземавме од ENTSO-E API содржеа празни (NaN) вредности за одредени часови. Тоа го решивме со **time-based интерполација**, односно оние часови кои недостасуваат, наместо просто копирање на претходната вредност, **method="time"** ја зема во предвид точната временска оддалеченост помеѓу познатите точки и пресметува средна вредност меѓу нив.

```
10 nan_before = df.iloc[:, 0].isna().sum()
11 df = df.interpolate(method="time")
12 nan_after = df.iloc[:, 0].isna().sum()
```

- За секоја земја собравме колку вкупно PV енергија (во TWh) е произведена за цела 2023 година и ја споредивме со реалните (јавно објавени) бројки за таа земја. Оваа проверка ја направивме пред да продолжиме понатаму со моделите, за да бидеме сигурни дека градиме на точни податоци.

```

43 nL_year = df[(df["datetime"] >= "2023-01-01") & (df["datetime"] < "2024-01-01")]
44 total_twh = nL_year["solar_generation_MW"].sum() / 1_000_000
    
```

RQ1 - Предвидување на цената на електричната енергија

- Правиме проценка колку прецизно може да се предвидат идните цени на електрична енергија со користење на **историски пазарни податоци** (историски цени: пред 1 час, 24 часа и 168 часа), **метеоролошки променливи** (температура, ветер, облачност, сончево зрачење) и **карактеристики за фотоволтаично производство** (lags за историско производство на фотоволтаици).

- Она што го правиме во овој дел опфаќа: тренирање и евалуација на модели за предвидување, споредба на различни конфигурации на карактеристики и евалуација со **MAE, RMSE, R²**.

- Целиот проект (AI-driven forecasting + battery optimization) зависи од две работи: **колку добро можеме да ја предвидиме цената (RQ1) и колку добро можеме да го предвидиме производството (RQ2)**. Профитот во battery trading директно зависи од точноста на ценовните сигнали - ако не знаеме кога е скапо/евтино, немаме основа за одлука.

- Пред да почнеме со сложени модели, направивме едноставен **“наивен”** модел кој само **претпоставува дека цената ќе остане иста и во иднина**. Ова е стандардна пракса (**секогаш треба да имаме база за споредба, за да дознаеме дали посложените модели ни носат некаква вредност**).

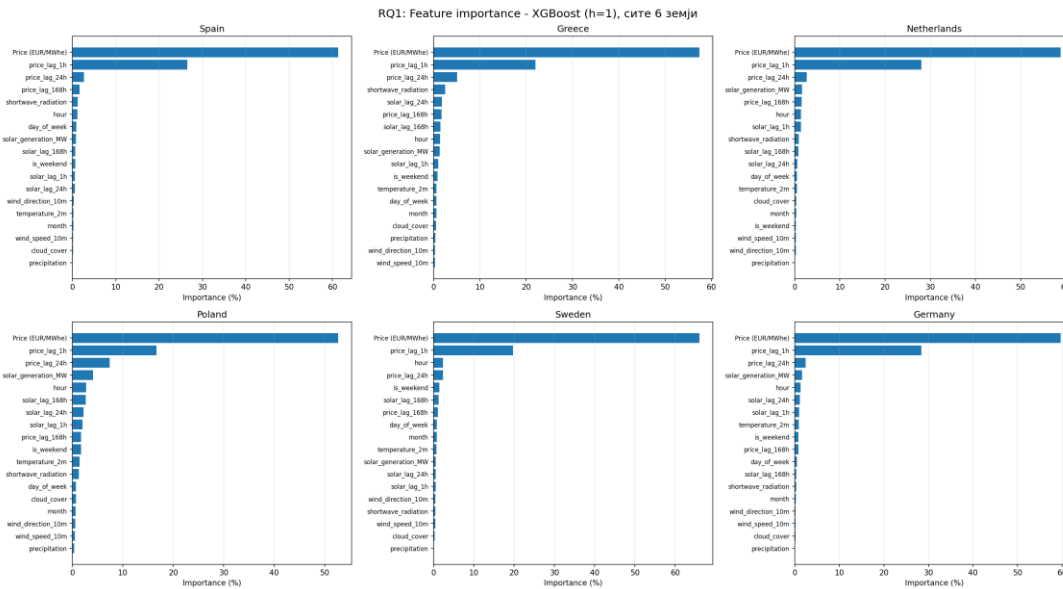
- Кај наивниот модел (**Baseline model**) дознавме дека е добар за предвидување (**79% точност**) за 1 час и 24 часа однапред, но многу лош во средината, пример 12 до 15 часа однапред. Кога почнавме да анализираме, сфативме дека ова има смисла бидејќи **цените на струјата имаат силен дневен циклус од 24 часа**, па затоа наивниот модел ги “фаќа” рабовите на тој циклус, ама не и средината.

```

19 for h in range(1, 25):
20     y_true = test[f"target_h{h}"]
21
22     y_pred = test["Price (EUR/MWhe)"]
23
24     mae = mean_absolute_error(y_true, y_pred)
25     rmse = mean_squared_error(y_true, y_pred) ** 0.5
26     r2 = r2_score(y_true, y_pred)
27
28     results.append({"horizon": h, "MAE": mae, "RMSE": rmse, "R2": r2})
    
```

- Втор модел за тренирање е **XGBoost** моделот. За **features** му дадовме **историски цени** (1 час, 24 часа и 168 часа) - овие бројки произлегоа од анализа на автокорелација која покажа јасни пикови на тие точки, со што се потврди **дневен и неделен циклус во податоците**. Освен тоа, имаме **календарски информации како час од денот и ден од неделата, метеоролошки услови и историско производство на фотоволтаици**. Овој модел достигна **88.6% точност**, што е

значително подобро од наивниот модел. Кога погледнавме кои информации моделот најмногу ги користи, откривме дека тековната цена и цената пред еден час заедно носат речиси **88% од целото знаење на моделот** додека временските услови придонесуваат со само околу 2%.



- После ова, сакавме да видиме дали посложени модели од претходните два ќе дадат уште подобри резултати, па продолживме со **LSTM (Long Short-Term Memory)**. Наместо да гледаме само неколку lag-вредности како XGBoost, **LSTM зема цела секвенца од 24 последователни часа** (сите features - цени, време, производство) и низ неа поминува чекор по чекор, со што гради внатрешна меморија за тоа што се случувало во тие 24 часа, за да го предвиди 25-тиот час. Овој модел на крајот постигна **82% точност**, а главна причина за тоа е дека **XGBoost веќе директно гледа “цена пред 1 час” и “цена пред 24 часа”** како готови информации, па затоа е малку **понебрецизен од XGBoost** бидејќи тој мора да ги најде истите временски врски што таму ни се дадени готови.

- На крај го пробавме и најсложениот - **TFT (Temporal Fusion Transformer)** модел. За разлика од LSTM (кој чита секвенца чекор по чекор и памти преку внатрешна состојба), TFT користи **механизам наречен attention** (за секоја точка во иднина што ја предвидува, моделот гледа наназад низ целата историја и сам одлучува кои конкретни минати часови се **најрелевантни** за таа конкретна прогноза, наместо сите да ги тренира подеднакво). Гледа 168 часа наназад (цела недела) и предвидува сите 24 идни часа однапред, за разлика од XGBoost и LSTM кои предвидуваат само 1 час однапред. Иако овој модел е најсложен, сепак даде послаб резултат со **точност од 51.7%**

Земја	Baseline	XGBoost	LSTM	TFT
Шпанија	85%	93.6%	90.1%	66.1%

Грција	73.2%	84.9%	77.7%	63.7%
Холандија	78.2%	88.4%	79.5%	48.6%
Полска	71.3%	83.7%	76.6%	42.9%
Шведска	87.6%	91.8%	84.8%	39.8%
Германија	79.8%	89.4%	84.1%	49.1%
Просек	79.2%	88.6%	82.1%	51.7%

- ЗАКЛУЧОК:

1. Кога ги видовме сите резултати заедно, дојдовме до јасен заклучок дека XGBoost е секаде најдобар модел во сите земји кои ги тестиравме (Шпанија, Грција, Холандија, Полска, Шведска, Германија). Редоследот е секаде ист: **XGBoost > LSTM > TFT**.

2. Направивме проверка дали XGBoost моделот имал overfitting, но резултатите на тренинг и тест сетот беа блиски (разлика од само 4-11 процентни поени по земја), што значи дека моделот навистина научил шаблони, не само специфични примери. **Поради тоа што XGBoost се покажа како најдобар модел, тој се користи и натаму во RQ3 (неговите предвидувања за цена влегуваат како влез во стратегијата за складирање/продажба на енергија понатаму во проектот).**

3. На крај, направивме и дополнително тестирање, да увидиме дали поголемите пазари (со поголемо PV производство) се полесин за предвидување. Резултатот беше негативен, **не постои таква врска**. Шведска (најмал производител) и Шпанија (еден од најголемите) имаа највисока точност, додека средните по големина (Полска, Грција) беа најтешки. **Ова значи дека веројатно не самата големина на пазарот, туку нивото и волатилноста на целите се она што реално ја определува предвидливоста.**

4. **Објаснување за слабиот резултат на TFT:** имавме сомнежи дека причината е тоа што веќе му дадовме готови lag features (price_lag_1h/24h/168h), а предноста на овој механизам е способноста сам да открие кои минати временски точки се важни, но кога веќе му ги имаме дадено дека нема шанса да покаже дополнителна вредност.

- За да провериме дали ова навистина е така, направивме ист **TFT модел но ги тргнавме lag features-от кои мислевме дека се проблемот**. Се покажа дека ни ова не е проблем, туку станува збор за **нефер споредба** по тежина на задачата (TFT мора да ги предвиди сите 24 хоризонти истовремено, па така овој 51.7% точност ни претставува просек преку сите тие лесни и тешки случаи (од h=1 до h=24)).

- Исто така резултатот веројатно е послаб бидејќи на моделот не му се дадени доволно податоци, но и дека **ваквиот тип на проблем не ја достигнува сложеноста** за ваков сложен модел да може да работи така како што треба.

Земја	TFT + lag	TFT - lag
Шпанија	66.1%	61.2%
Грција	63.7%	59%
Холандија	48.6%	37.4%
Полска	42.9%	36%
Шведска	39.8%	41.6%
Германија	49.1%	43.6%
Просек	51.7%	46.5%

RQ2 - Предвидување на фотоволтаично производство

- Правиме проценка **колку ефикасно можат метеоролошките податоци** (температура, ветер, облачност, сончево зрачење) **да го предвидат производството на фотоволтаична енергија, користејќи историско производство** (lags: 1 час, 24 часа, 168 часа) **и календарски информации**. Моделите тука предвидуваат вистинско идно производство ($h=1$ час однапред).
- Она што го правиме во овој дел опфаќа: тренирање и евалуација на модели за предвидување (истите како кај RQ1) и евалуација со MAE, RMSE, R2.
- Овој дел е важен затоа што одлуките за складирање во RQ3 директно зависат од тоа колку добро знаеме **колку енергија ќе биде произведена** - ако предвидувањето на производството е лошо, целата battery-optimization стратегија се гради врз погрешна основа.
- Прво направивме “наивен” (**Baseline**) модел - тековното производство директно како “предвидување” за h часа однапред. Овој модел беше солиден (85-93% точност за повеќето земји за $h=1$), бидејќи PV производството е силно циклично - следи ист дневен шаблон секој ден во зависност од положбата на Сонцето.

```

19     for h in range(1, 25):
20         y_true = test[f"target_h{h}"]
21
22         y_pred = test["Price (EUR/MWhe)"]
23
24         mae = mean_absolute_error(y_true, y_pred)
25         rmse = mean_squared_error(y_true, y_pred) ** 0.5
26         r2 = r2_score(y_true, y_pred)
27
28         results.append({"horizon": h, "MAE": mae, "RMSE": rmse, "R2": r2})
    
```

- Втор модел е **XGBoost**, со features: **час од денот, ден од неделата, месец, температура, ветер, облачност, зрачење и историско фотоволтаично производство**. За разлика од Baseline, овој модел **гради 24 одделни модели по земја** - еден специјализиран модел за секој хоризонт $h=1, \dots, 24$. Постигна просечно повисока точност од Baseline кај повеќето земји и хоризонти. Она што е интересно тука е тоа што, за **Холандија**, подобар модел беше Baseline, и ова можеби **покажува дека за потешки/понестабилни пазари, едноставните модел можат да бидат конкуренти**. За овој случај проверивме дали има overfitting, но разликата меѓу тренинг и тест податоците беше многу мала, што увидовме дека овде нема учење на памет, туку легитимен резултат: **PV производството е скоро детерминистичко (директно зависи од физичката положба на Сонцето), многу попредвидливо од цените**.

```

7     pv_features = [
8         'hour', 'day_of_week', 'month', 'is_weekend',
9         'temperature_2m', 'wind_speed_10m', 'cloud_cover', 'shortwave_radiation',
10        'solar_lag_1h', 'solar_lag_24h', 'solar_lag_168h'
11    ]
12
13    MODEL_PARAMS = dict(
14        n_estimators=300, max_depth=3, learning_rate=0.05,
15        min_child_weight=5, subsample=0.8, colsample_bytree=0.8,
16        reg_alpha=1, reg_lambda=1, random_state=42
17    )
    
```

```

24 train_end = int(len(df) * 0.7)
25 val_end = int(len(df) * 0.85)
26
27 X_train = df[pv_features].iloc[:train_end]
28 X_test = df[pv_features].iloc[val_end:]
29
30 results = []
31 predictions = pd.DataFrame(index=df.index[val_end:])
32 for h in range(1, 25):
33     y_train = df[f"target_pv_h{h}"].iloc[:train_end]
34     y_test = df[f"target_pv_h{h}"].iloc[val_end:]
35
36     model = XGBRegressor(**MODEL_PARAMS)
37     model.fit(X_train, y_train)
38     y_pred = model.predict(X_test)
39     predictions[f"predicted_P_h{h}"] = y_pred
40
41     mae = mean_absolute_error(y_test, y_pred)
42     rmse = mean_squared_error(y_test, y_pred) ** 0.5
43     r2 = r2_score(y_test, y_pred)
    
```

- Важно да се напомене е дека во почетната фаза на проектот (со мало податочно множество од само 3 месеци и 1 земја), **XGBoost даде негативен резултати (-3.3%) поради distribution shift** (тренинг периодот беше лето - што значи високо производство, а тест период беше есен - пониско производство), па моделот систематски преценуваше. **Со проширеното податочно множество (6 земји и период од 4 години), овој проблем исчезна.**

- Трет модел е **LSTM**, кој работи слично како во RQ1 - гледа 24-часовна секвенца и учи внатрешна меморија, повторно со по 24 одделни модели за секој хоризонт. Постигна највисока точност од сите три (h=1), со многу стабилна деградација низ хоризонтот - потврда дека **PV производството останува предвидливо дури и 24 часа однапред.**

- За крај, **TFT** моделот - истата attention базирана архитектура како во RQ1, гледа 168 часа наназад и предвидува сите 24 часа однапред истовремено (нема 24 одделни модели). Постигна просечно **85.4% точност**, послабо од XGBoost и LSTM, со Холандија повторно најтежок случај (само 64.1%).

Земја	Baseline (h=1)	XGBoost (h=1)	LSTM (h=1)	TFT
Шпанија	88.4%	96.0%	98.1%	92.1%
Грција	89.2%	96.4%	97.9%	88.5%
Холандија	85.2%	83.8%	85.0%	64.1%
Полска	91.2%	96.5%	97.7%	88.9%
Шведска	92.5%	97.0%	98.9%	86.9%
Германија	91.6%	97.5%	98.9%	91.8%
Просек	89.7%	94.5%	96.1%	85.4%

- ЗАКЛУЧОК:

1. **LSTM е најдобар модел** на краток хоризонт (h=1) со просек од **96.1%**, потоа по него следи XGBoost (**94.5%**) - за разлика од RQ1. Тука редоследот е **LSTM > XGBoost > TFT**.

2. **PV производството е многу попредвидливо од цени**, дури и на вистински хоризонт (83-98% наспроти 40-90% за цени во RQ1) - бидејќи е поврзано со физички закони (позиција на Сонце).

3. **Холандија е конзистентно најтежок случај** за секој модел, а разликите меѓу моделите таму се најмали (83.8% - 85.2%) - веројатно поради поради понестабилно производство кое го нарушува дневниот шаблон на кој се потпираат сите модели.

4. **Distribution shift**: доволен временски опфат на податоците (4 години наспроти 3 месеци) целосно го решава distribution-shift проблемот кој го имавме во почетната фаза.

5. **Методолошка забелешка**: Baseline моделот покажува **екстреман U-облик по хоризонт** (одличен на $h=1$ и $h=24$, но многу лош во средината - R2 доаѓа и до -255% на $h=12$), бидејќи **PV производството оди од нула (ноќе) до максимум (напладне) и назад**, па затоа $h=1$ споредба е поинформативна од просек преку сите хоризонти за овој модел.

6. Иако LSTM е за 1.6% подобар, сепак го избравме XGBoost за конзистентност и практичност, бидејќи и во RQ1, XGBoost е најдобар.

RQ3 - Оптимизација на складирањето на енергија во батерија

- Прашањето на кое одговараме е **која стратегија за складирање и продажба на енергија го максимизира приходот**, кога идните цени и идното производство не се точно познати, туку **само проценети преку модели**.

- За да одговориме на ова прашање, **создадовме ново податочно множество**, во кое ги ставивме **предвидувањата за цена од RQ1 и предвидувањата за производство од RQ2 со податоците кои ги добивме од најдобрите модели според резултатите** (во RQ2, разликата помеѓу перформансите на XGBoost и LSTM беше 1.6%, иако LSTM беше подобар, го земавме и тука XGBoost бидејќи тој беше најдобар и во RQ1 - најмногу за конзистентност и не мешање на архитектури). **Со ова ја потврдуваме слоевитата структура на проектот**. Конкретно, за секој час t од test периодот, ги земавме: вистинската цена и производство во моментот t , плус 24 предвидувања за цена ($h=1$ до $h=24$) од RQ1 и 24 предвидувања за производство ($h=1$ до $h=24$) од RQ2 - вкупно 48 предвидени вредности + тековните, кои заедно ја формираат оваа табела (rq3_inputs) врз која се основаат трите стратегии (Rule-based, MPC, Perfect Foresight).

- За секој час t , **MPC-то гледа 24-часовен прозорец однапред - позицијата “сега” (t) користи вистинска, позната вредност (тековна цена, тековно производство), а позициите $t+1, \dots, t+23$**

користат хоризонт-специфични прогнози, истата методологија од RQ1/RQ2 (24 посебни модели, еден за секој хоризонт).

- Battery моделот што го користиме: **капацитетот на батеријата е пропорционален на просечното (train) производство на земјата = 2x просечно производство**, за секоја земја да добие реалистично скалирана батерија (Германија со огромно производство добива капацитет од 14 043, Холандија со мало производство добива само 124).

- **Батерија работи само меѓу Min SOC=10% и Max SOC=90%** од капацитетот (не 0-100%), бидејќи **целото празнење/полнење ги оштетува литиум-јонските батерии** побрзо и им го скратува животниот век (стандардна индустриска препорака).

- **Максималната брзина на полнење/празнење е 40% од капацитетот на час** (целосно полнење/празнење бара **минимум 2.5 часа** - ова е реалистична брзина за мрежни батерии)

- **Ефикасноста на полнење/празнење е 95%**, типична вредност за модерни литиум-јонски батерии (**секое полнење/празнење губи мал дел енергија како топлина**).

- **Rule-based** стратегијата не прави никаква вистинска пресметка/оптимизација, **само споредува бројки**. За секој час, гледа дали тековната (познатата) цена е над однапред пресметан праг (75-ти перцентил од train), ако е - **продава/празни батерија**, а ако е под 25-ти перцентил, **полни батерија**. Нема никаква интелигенција, **само if-else логика и затоа ни служи како baseline** (секое подобрување над ова докажува дека посложениот пристап носи некаква вредност).

```

21 low_price = train['Price (EUR/MWhe)'].quantile(0.25)
22 high_price = train['Price (EUR/MWhe)'].quantile(0.75)
23
24 soc = MIN_SOC
25 revenue = 0
26
27 for i, row in df.iterrows():
28     production = row['actual_P']
29     real_price = row['actual_price']
30
31     charge = 0
32     discharge = 0
33     sell = production
34
35     if real_price > high_price:
36         discharge = min(MAX_RATE, soc - MIN_SOC)
37     elif real_price < low_price:
38         charge = min(production, MAX_RATE, MAX_SOC - soc)
39         sell = production - charge
40
41     revenue += real_price * (sell + discharge * EFFICIENCY) / 1000
42     soc = soc + charge * EFFICIENCY - discharge
    
```

- **MPC** стратегијата пресметува **преку Linear Programming** - математичка техника која наоѓа **најдобра комбинација на броеви** (колку да се продаде, колку да се наполни, колку да се испразни батеријата секој час) која **го максимизира вкупниот приход**, притоа да почитува **точно дефинирани ограничувања**.

- За секој 24-часовен прозорец, солверот (PuLP/CBC) бара вредности за секој час t кои: (1) го

максимизираат вкупниот приход (цена $\times$ продадена + испразнета енергија), (2) обезбедуваат дека целото произведено количество некаде оди (продажба/полнење), (3) го следат точното менување на состојбата на батеријата час по час и (4) не дозволуваат батеријата да прелее преку максимумот или да падне под минимумот, ниту да се полни/празни побрзо од дозволената брзина. Од целото решение за 24-те часа, се извршува само одлуката за првиот час (со вистински вредности), потоа прозорецот се поместува за 1 час и истата постапка се повторува, па затоа и се вика **Model Predictive Control**, секој пат ре-планира со најнови информации.

```

12 def solve_lp_window(prices, productions, soc_start, max_soc, min_soc, max_rate):
13     H = len(prices)
14     prob = pulp.LpProblem(name="mpc_window", pulp.LpMaximize)
15     sell = [pulp.LpVariable(name=f"sell_{t}", lowBound=0) for t in range(H)]
16     charge = [pulp.LpVariable(name=f"charge_{t}", lowBound=0, upBound=max_rate) for t in range(H)]
17     discharge = [pulp.LpVariable(name=f"discharge_{t}", lowBound=0, upBound=max_rate) for t in range(H)]
18     soc = [pulp.LpVariable(name=f"soc_{t}", lowBound=min_soc, upBound=max_soc) for t in range(H)]
19     prob += pulp.lpSum(prices[t] * (sell[t] + discharge[t] * EFFICIENCY) for t in range(H))
20     for t in range(H):
21         prob += sell[t] + charge[t] == productions[t]
22         prev_soc = soc_start if t == 0 else soc[t - 1]
23         prob += soc[t] == prev_soc + charge[t] * EFFICIENCY - discharge[t]
24     prob.solve(pulp.PULP_CBC_CMD(msg=0))
25     return sell[0].varValue, charge[0].varValue, discharge[0].varValue
    
```

```

50     for t in range(n):
51         row = df.iloc[t]
52         remaining = min(HORIZON, n - t)
53
54         future_prices = row[price_cols].values[:remaining - 1]
55         future_prod = row[pv_cols].values[:remaining - 1]
56
57         window_prices = [row['actual_price']] + list(future_prices)
58         window_prod = [row['actual_P']] + list(future_prod)
59
60         sell, charge, discharge = solve_window(
61             window_prices, window_prod, soc, MAX_SOC, MIN_SOC, MAX_RATE
62         )
63
64         real_price = df['actual_price'].iloc[t]
65         real_production = df['actual_P'].iloc[t]
66
67         actual_sell = min(sell, real_production)
68         actual_charge = min(charge, real_production - actual_sell, MAX_SOC - soc)
69         actual_discharge = min(discharge, soc - MIN_SOC)
70
71         surplus = real_production - actual_sell - actual_charge
72         actual_sell += surplus
73
74         revenue += real_price * (actual_sell + actual_discharge * EFFICIENCY) / 1000
75         soc = soc + actual_charge * EFFICIENCY - actual_discharge
    
```

- **Perfect Foresight** ја користи истата LP логика како MPC, со единствена разлика, наместо прогнозирани идни вредности за цена и производство, во прозорецот се внесуваат вистинските, реални идни вредности (кои во реалноста не би биле познати однапред). Ова ни дава теоретски максимален можен приход, ни служи како горна граница за споредба.

```

12 def solve_lp_window(prices, productions, soc_start, max_soc, min_soc, max_rate):
13     H = len(prices)
14     prob = pulp.LpProblem(name="mpc_window", pulp.LpMaximize)
15     sell = [pulp.LpVariable(name=f"sell_{t}", lowBound=0) for t in range(H)]
16     charge = [pulp.LpVariable(name=f"charge_{t}", lowBound=0, upBound=max_rate) for t in range(H)]
17     discharge = [pulp.LpVariable(name=f"discharge_{t}", lowBound=0, upBound=max_rate) for t in range(H)]
18     soc = [pulp.LpVariable(name=f"soc_{t}", lowBound=min_soc, upBound=max_soc) for t in range(H)]
19     prob += pulp.lpSum(prices[t] * (sell[t] + discharge[t] * EFFICIENCY) for t in range(H))
20     for t in range(H):
21         prob += sell[t] + charge[t] == productions[t]
22         prev_soc = soc_start if t == 0 else soc[t - 1]
23         prob += soc[t] == prev_soc + charge[t] * EFFICIENCY - discharge[t]
24     prob.solve(pulp.PULP_CBC_CMD(msg=0))
25     return sell[0].varValue, charge[0].varValue, discharge[0].varValue

```

```

45     for t in range(n):
46         window = df.iloc[t:t+HORIZON]
47         sell, charge, discharge = solve_window(
48             window['actual_price'].values, window['actual_P'].values, soc, MAX_SOC, MIN_SOC, MAX_RATE
49         )
50
51         real_price = df['actual_price'].iloc[t]
52         real_production = df['actual_P'].iloc[t]
53
54         actual_sell = min(sell, real_production)
55         actual_charge = min(charge, real_production - actual_sell, MAX_SOC - soc)
56         actual_discharge = min(discharge, soc - MIN_SOC)
57
58         surplus = real_production - actual_sell - actual_charge
59         actual_sell += surplus
60
61         revenue += real_price * (actual_sell + actual_discharge * EFFICIENCY) / 1000
62         soc = soc + actual_charge * EFFICIENCY - actual_discharge

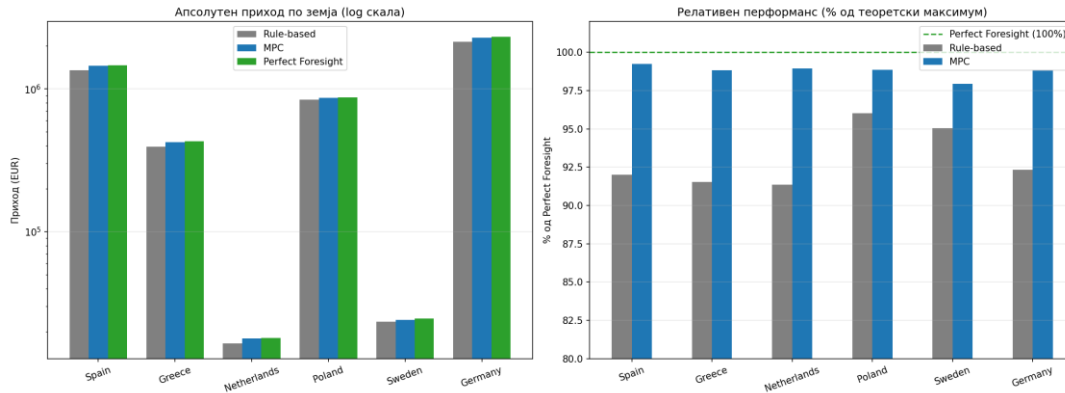
```

- Резултати:

Земја	Rule-based	MPC	Perfect Foresight
Шпанија	1.348.588 EUR	1.454.609 EUR	1.465.655 EUR
Грција	392.580 EUR	423.891 EUR	428.938 EUR
Холандија	16.519 EUR	17.886 EUR	18.079 EUR
Полска	838.562 EUR	863.483 EUR	873.354 EUR
Шведска	23.429 EUR	24.139 EUR	24.646 EUR
Германија	2.134.283 EUR	2.285.337 EUR	2.311.480 EUR

Земја	Rule-based	MPC
Шпанија	92%	99.2%
Грција	91.5%	98.8%
Холандија	91.4%	98.9%

Полска	96%	98.9%
Шведска	95.1%	97.9%
Германија	92.3%	98.9%



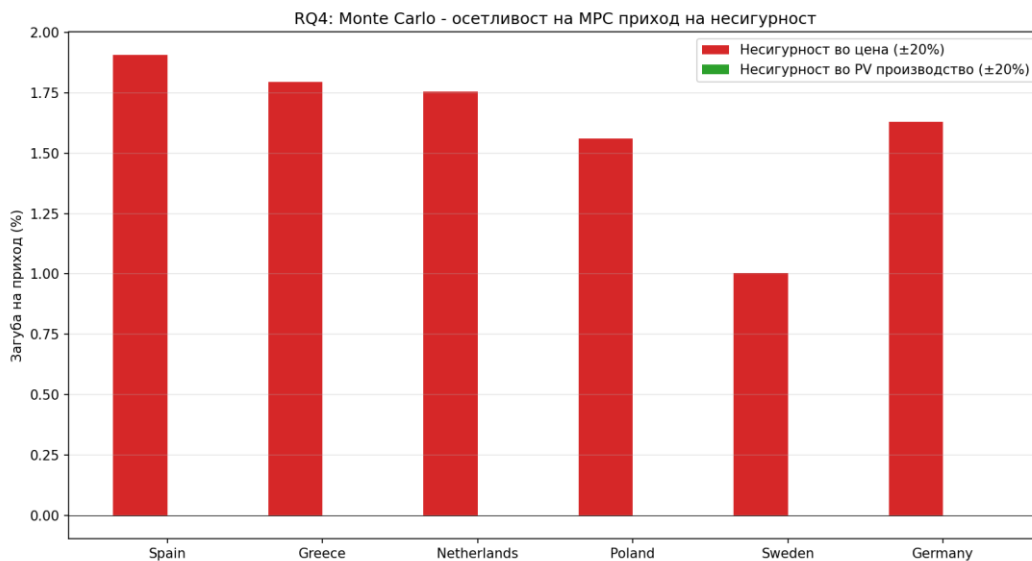
- ЗАКЛУЧОК:

- 1. MPC е подобар од rule-based за секоја земја**, со што се потврдува дека формалната оптимизација носи реална вредност над едноставните (rule-based) прагови.
- 2. MPC постигнува многу близу до теоретски оптимално**, дури и со реалистични, несовршени прогнози, **се губи само 1-2% од максималниот можен приход**.
- 3. Батеријата систематски завршува на минимум SOC на крајот на секој 24-часовен прозорец на планирање ("terminal SOC bias")** - MPC-от нема причина да "чува" енергија за после видливиот хоризонт, па резултатите се малку оптимистични.
- 4. Проектот е слоевит**, не само методолошки сличен, тоа значи дека **RQ3 директно ги презема веќе тестираните и потврдени предвидувања од RQ1 и RQ2** (48 хоризонт-специфични прогнози по час), наместо повторно да генерираме сопствени. Ова значи дека **секое подобрување или грешка во RQ1/RQ2 директно се пренесува во RQ3**, токму затоа **беше важно овие два модели прво да се валидираат одделно, пред нивните предвидувања да се употребат во оптимизацијата**.

RQ4 - Влијание на неизвесноста врз стратегијата

- Прашањето на кое одговараме е колку е чувствителна MPC стратегијата на грешки во предвидувањето и дали грешките во некои услови (облачно, викенд, зима) чинат повеќе отколку во други.

- Прв тест кој го направивме е **Monte Carlo симулација: генериравме случаен шум ($\pm 20\%$ стандардна девијација)** околу прогнозите за цена и за производство, одделно, 10 повторувања за секоја и мериме колку паѓа приходот споредено со base MPC приходот без шум.



- Она што го дознавме тука е тоа што **несигурноста во PV производството е практично безначајна за приходот**, додека **несигурноста во цената дава конзистентна загуба** (во просек околу 1.61%). Ова значи дека **инвестирањето во подобрувањето на предвидувањето на цени (RQ1) би донело поголема практична вредност отколку подобрување во предвидувањето на PV производството (RQ2).**

- Втор тест кој го направивме е **систематски (fixed) нарушувања**: наместо да додаваме случаен шум, **систематски ги скалираме идните предвидени цени со фиксен процент ($\pm 10\%$, $\pm 20\%$, $\pm 30\%$)** и мериме како ќе реагира приходот. Резултатот покажа hump облик, **негативно скалирање (ги потценува идните цени), секогаш дава повисок приход од позитивното скалирање (преценување) со иста големина.**

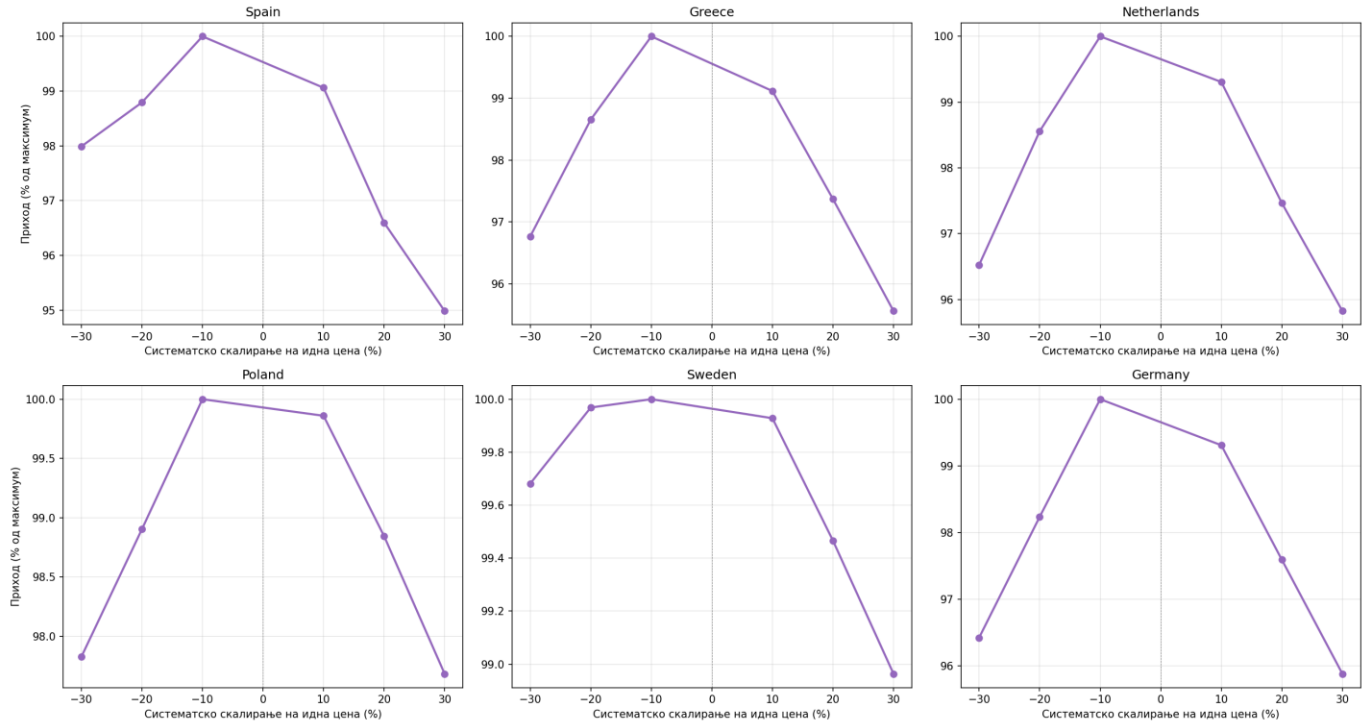
- Објаснување: кога MPC потценува идни цени, тој е претпазлив и продава веднаш по позната, сигурна тековна цена, а кога преценува, тој складира енергија со тоа што очекува повисоки идни цени кои во реалноста не би се материјализирале толку силно.

Земја	Загуба - несигурност во цена	Загуба - несигурност во производство
Шпанија	1.91%	~0%

Грција	1.8%	~0%
Холандија	1.76%	~0%
Полска	1.56%	~0%
Шведска	1%	~0%
Германија	1.61%	~0%

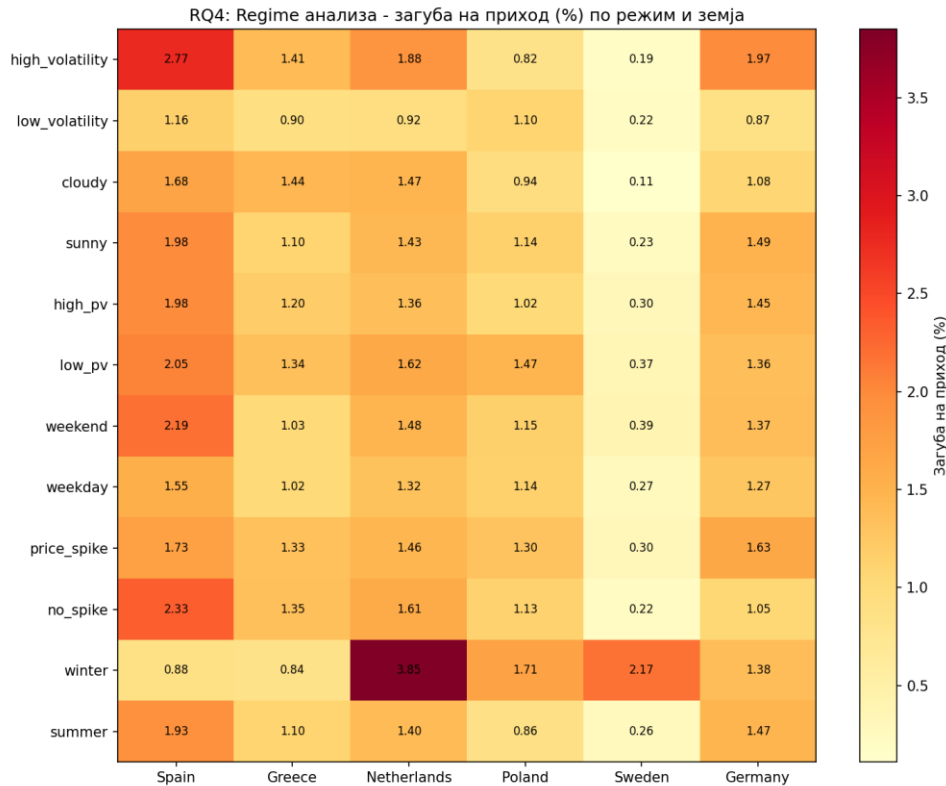
Земја	-30%	-20%	-10%	+10%	+20%	+30%
Шпанија	1,404,872	1,427,217	1,447,536	1,436,618	1,386,089	1,356,260
Грција	407,039	418,683	423,920	419,381	403,168	392,507
Холандија	17,143	17,608	17,842	17,687	17,302	16,872
Полска	830,841	845,017	859,103	854,538	837,456	824,360
Шведска	23,572	23,741	23,985	23,982	23,576	23,334
Германија	2,198,609	2,249,049	2,281,165	2,265,096	2,216,204	2,157,183

RQ4: Асиметрична чувствителност на MPC (преценување vs потценување на идни цени)



- Трет тест кој го направивме е **анализа по режими**: го поделивме тест периодот на парови од спротивни режими (**висока/ниска волатилност, облачно/сончево, високо/ниско производство, викенд/работен ден, price spike/по spike, зима/лето**) и мериме % загуба на приход одделно за секој пар.

Земја	High vol.	Low vol.	Cloudy	Sunny	High PV	Low PV	Weekend	Weekday	Price spike	No spike	Winter	Summer
Шпанија	2.64%	1.11%	1.75%	1.94%	1.78%	1.45%	1.78%	1.81%	1.77%	1.98%	1.51%	1.80%
Грција	2.47%	1.02%	2.35%	1.41%	1.27%	2.49%	1.64%	1.89%	1.97%	1.35%	1.77%	1.28%
Холандија	2.67%	1.15%	2.76%	1.43%	1.15%	3.66%	1.29%	1.83%	1.83%	1.23%	3.32%	1.37%
Полска	1.79%	1.21%	1.98%	1.30%	1.22%	2.18%	1.58%	1.32%	1.53%	1.45%	2.40%	1.27%
Шведска	1.48%	0.49%	1.41%	0.83%	0.47%	2.35%	0.63%	1.01%	1.15%	0.84%	1.58%	0.67%
Германија	2.48%	1.09%	2.02%	1.63%	1.71%	2.25%	1.00%	1.72%	1.84%	1.15%	1.45%	1.28%



- ЗАКЛУЧОК:

1. MPC стратегијата е асиметрично чувствителна на насоката на грешка - **преценување на идни цени е поскапа грешка од потценување** (истото е кај сите земји).
2. Несигурноста во PV производството е практично без значење, додека несигурноста во цената дава одредена загуба (околу 1.6%). Тоа ни е силна потврда дека **RQ1 (ценовното предвидување) е попродуктивна инвестиција од RQ2 (PV производство) за подобрување на трговската стратегија.**
3. **Висока волатилност е најголемиот ризик за сите земји подеднакво** (загубата е од 2.6 до 5.4 пати поголема отколку кога цената е стабилна).
4. **Холандија е најслаба точка во скоро секој случај**, најмногу зимно време (3.32% загуба, наспроти 1.37% летно време) и денови со малку Сонце (3.66% загуба, наспроти 1.15% кога има многу Сонце).
5. **Шведска е најотпорна земја, има најниски загуби во секој режим веројатно поради помал, но постабилен пазар кој нема нагли ценовни промени.**
6. **Препорака:** ако некој сака да ја подобри стратегијата против ризик, најголема вредност би имало подобрувањето на прогнозите специфично за: **нестабилни ценовни периоди, зимски месеци и денови со ниско производство.**

ПРОБЛЕМИ СО КОИ СЕ СООЧИВМЕ ПРИ РАБОТАТА НА ПРОЕКТОТ

1. Најголема грешка направивме во RQ3. Кога генериравме “предвидена цена” за секој час, ненамерно користевме вистинска цена од идниот час (информација која реално не би ја знаеле однапред). На пример, за да предвидиме цена 23 часа однапред, случајно погледнавме цена само 1 час пред таа. Го докажавме тоа со бројки: првата прогноза беше 93% толност, а откако го поправивме се намали на 51%. Го поправивме со тоа што направивме 24 засебни модели, еден за секој час однапред и секој од нив користи само информации што навистина ни се познати во моментот.
2. Втора грешка, праговите за rule-based стратегијата (кога цената е скапа или евтина) ги пресметувавме со погрешен дел од податоците (од тест сетот наместо од тренинг сетот). Тоа значи дека ние на моделот индиректно му дававме знаење за тоа какви ќе бидат идните цени. Тоа го поправивме со тоа што праговите ги направивме да доаѓаат само од тренинг податоците.
3. Кај LSTM моделот забележавме дека секој пат кога го пуштаме истиот код, добивавме различен резултат (70%, 88%). Она што беше проблемот е дека немаше фиксен почетен број за случајност (seed). Со додавање на фиксен seed, резултатите беа исти секој пат кога ќе го пуштем кодот.
4. Истиот проблем го имавме и со TFT моделот, исто така го поправивме и тоа.
5. Во RQ2 направивме грешка така што моделите првично предвидуваа тековно производство (гледаа временски услови сега, за да го погодат сегашното производство). Ова всушност не одговараше на прашањето и не предвидуваше идно производство.
6. Во почетните експерименти, XGBoost ни даде негативен резултат бидејќи имавме многу краток период (3 месеци) за само една земја, се јави distribution shift, кое при работење на вистинскиот проблем исчезна.

РАБОТИ ОД ПРЕДМЕТОТ КОИ ГИ ИМАМЕ ПРИМЕНЕТО ВО ПРОЕКТОТ

1. Собирање на податоци, правење на истражувачка анализа, читање на графици.
2. Операции врз податоците, спојување на податочни множества, изведување на нови карактеристики од веќе постоечките.
3. **Feature engineering** (lag/target карактеристики), dropna, train/test поделба, скалирање на податоците.
4. Секој модел (Baseline/ XGBoost, LSTM, TFT) е всушност **Supervised Learning Model**.
5. **Bagging and Boosting** преку XGBoost моделот.
6. **Невронски мрежи** преку LSTM и TFT моделите.
7. **Временски серии** - RQ1 и RQ2 се time series forecasting.

GitHub линк до репозиториумот: <https://github.com/hristinagjorgjievska/pv-energy-trading-optimization>